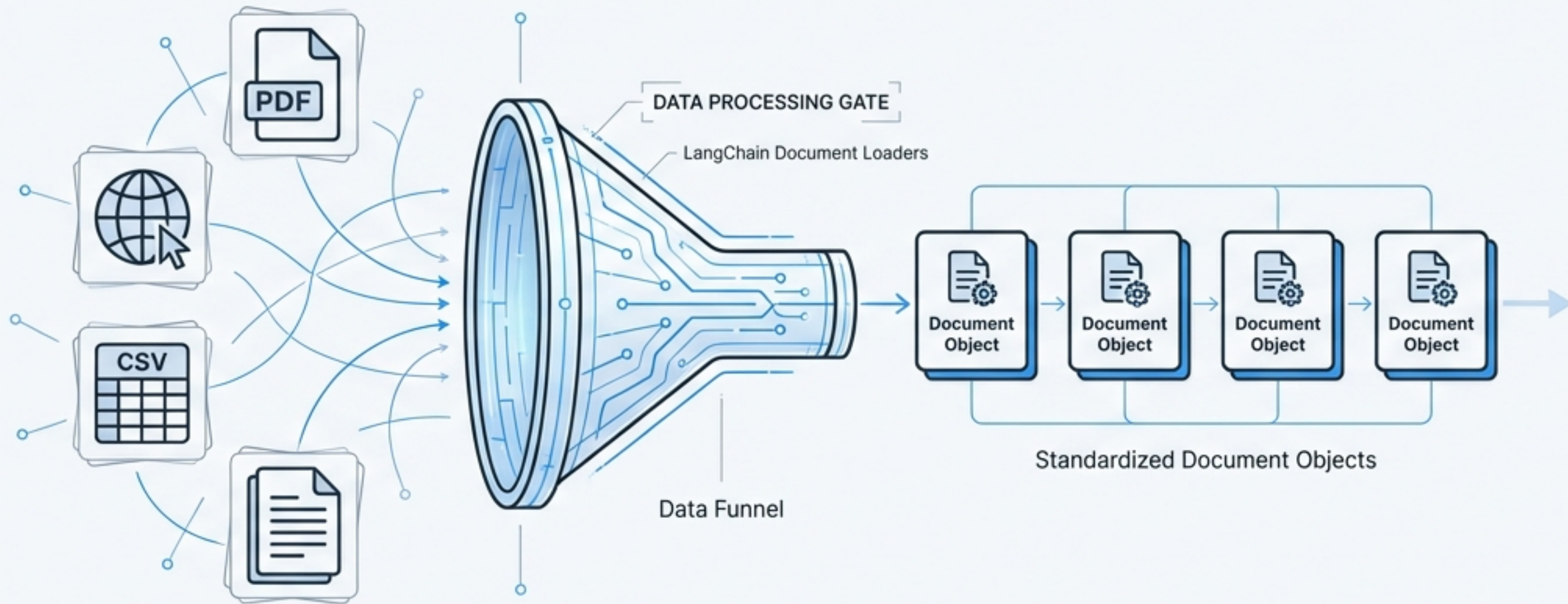


Mastering Document Loaders in LangChain

The critical first step in building Retrieval Augmented Generation (RAG) pipelines



Based on the LangChain technical series by CampusX

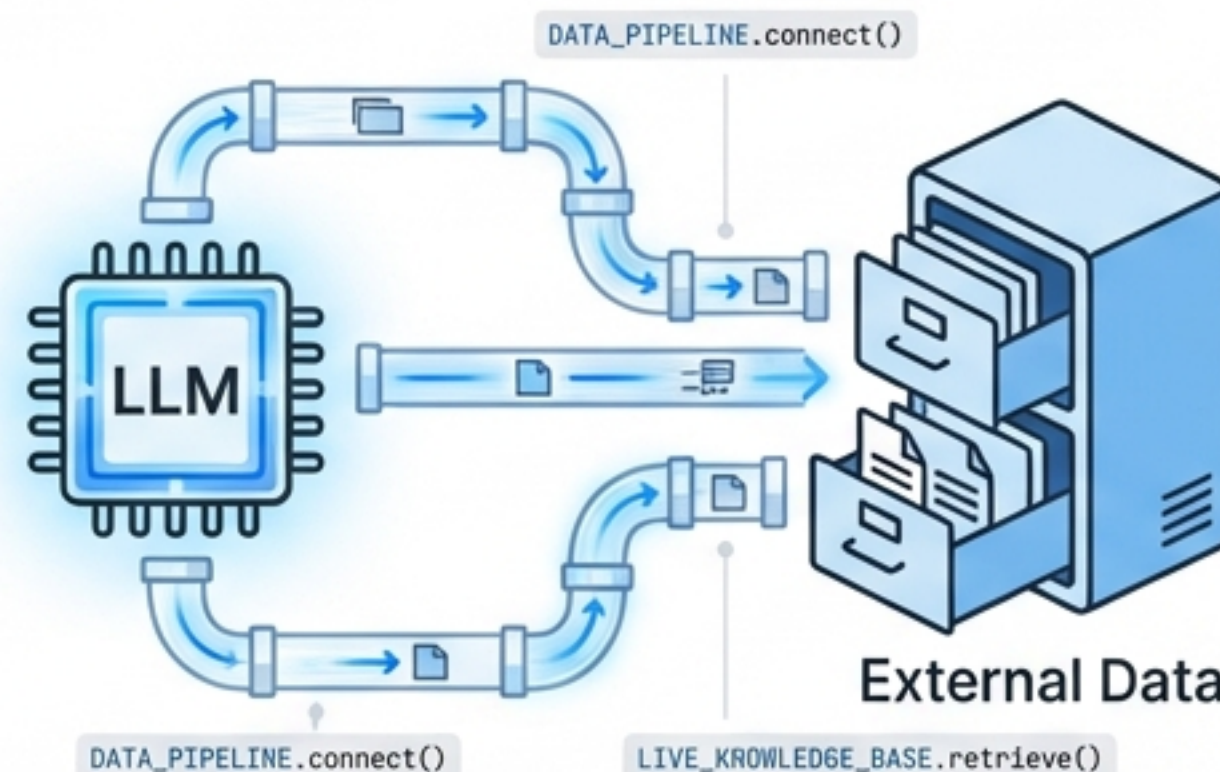
LLMs have a knowledge gap that RAG bridges

The Knowledge Gap



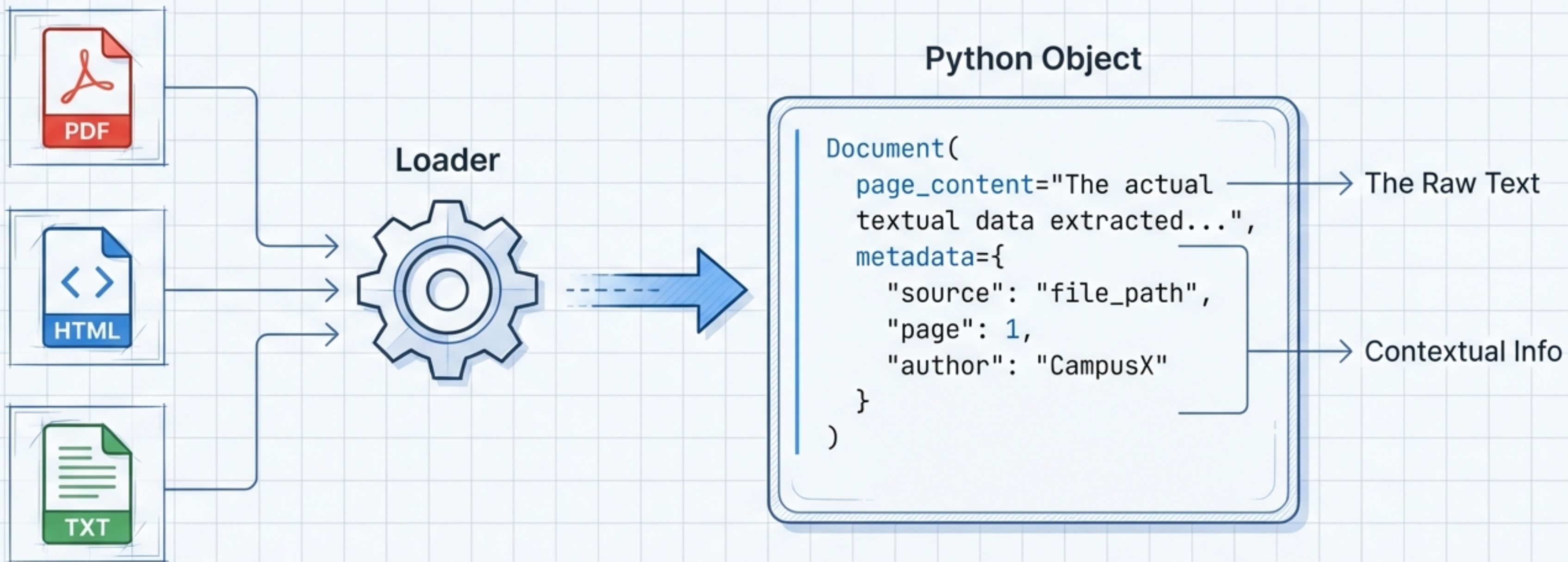
- ⇒ LLMs are frozen in time based on training data cut-offs.
- ⇒ No access to private/proprietary data (e.g., company emails, internal docs).

Retrieval Augmented Generation (RAG)



- Connects the LLM to a live external knowledge base.
- Provides grounded, up-to-date responses without model retraining.

Standardising chaos into the 'Document' Object



TextLoader establishes the baseline pattern

```
from langchain_community.document_loaders import TextLoader

# 1. Instantiate
loader = TextLoader('cricket.txt', encoding='utf-8')

# 2. Load
docs = loader.load()
```

Output: List[Document]

Document

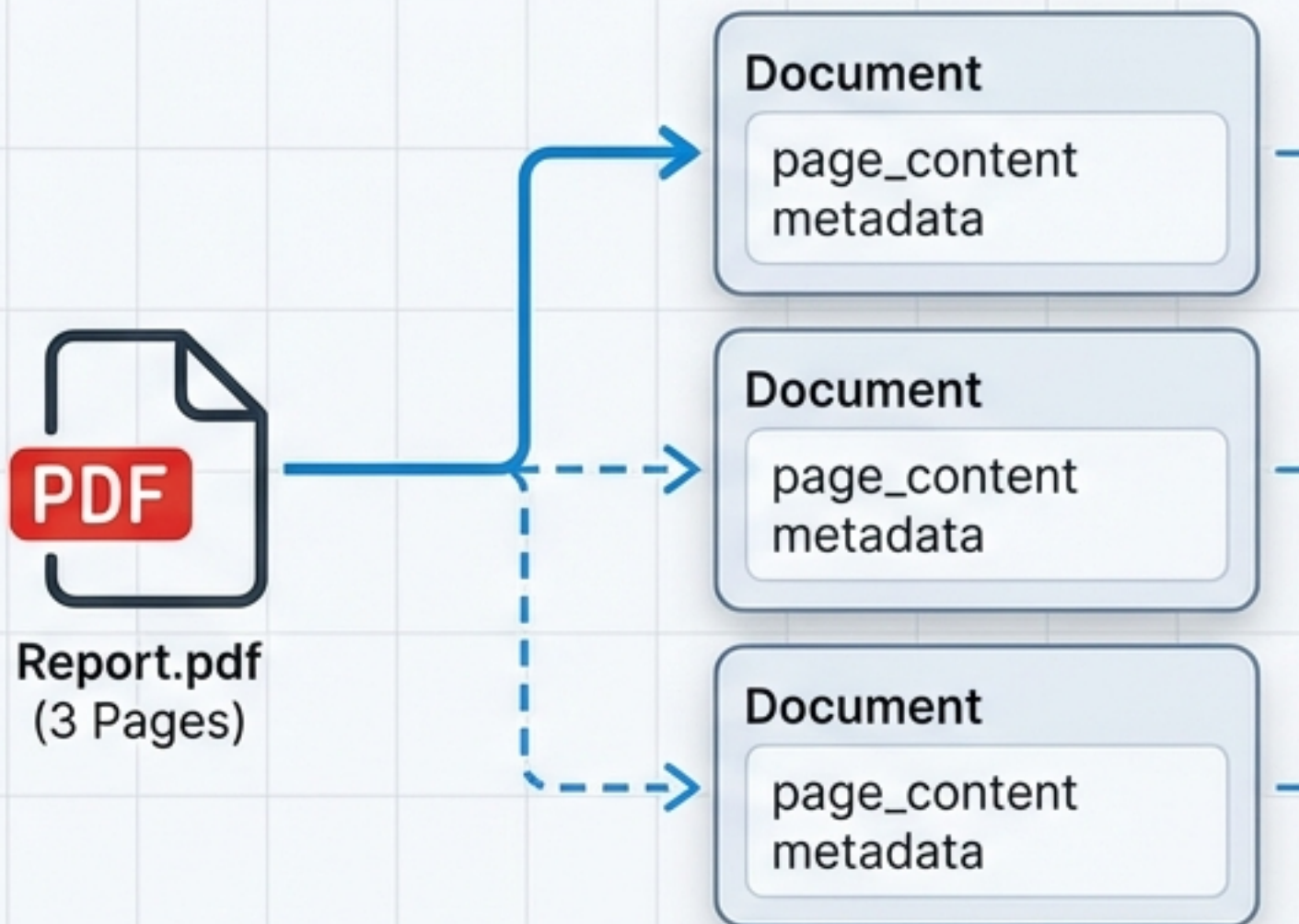
page_content="Match commentary..."
metadata={'source': 'cricket.txt'}

Returns a list, even
for single files.

PyPDFLoader handles page-by-page splitting

Concept

Code



```
from langchain_community.document_loaders import
    PyPDFLoader

loader = PyPDFLoader('DL_Curriculum.pdf')
docs = loader.load()

# Result: len(docs) == 23 (if PDF has 23 pages)
```



Pro Tip: PyPDFLoader uses `pypdf` and is best for selectable text. For scanned images or tables, use `PDFPlumber`, `UnstructuredPDFLoader`, or `AmazonTextractPDFLoader`.

DirectoryLoader scales ingestion to entire folders



```
from langchain_community.document_loaders import DirectoryLoader

# Load all PDFs in the 'books' directory
loader = DirectoryLoader(
    'books',
    glob='*.pdf',
    loader_cls=PyPDFLoader
)

docs = loader.load()
```

DirectoryLoader



glob Pattern matching (e.g., `\*.pdf` filters only PDF files).



loader_cls Delegates the actual parsing to a specific class (e.g., PyPDFLoader).

Aggregates all pages from all found files into a single flat list.

Optimise memory usage with Lazy Loading

Eager Loading (`` .load()``)



```
docs = loader.load()
```

- Returns a List
- High Memory Usage
- Blocks until done
- Good for small datasets.

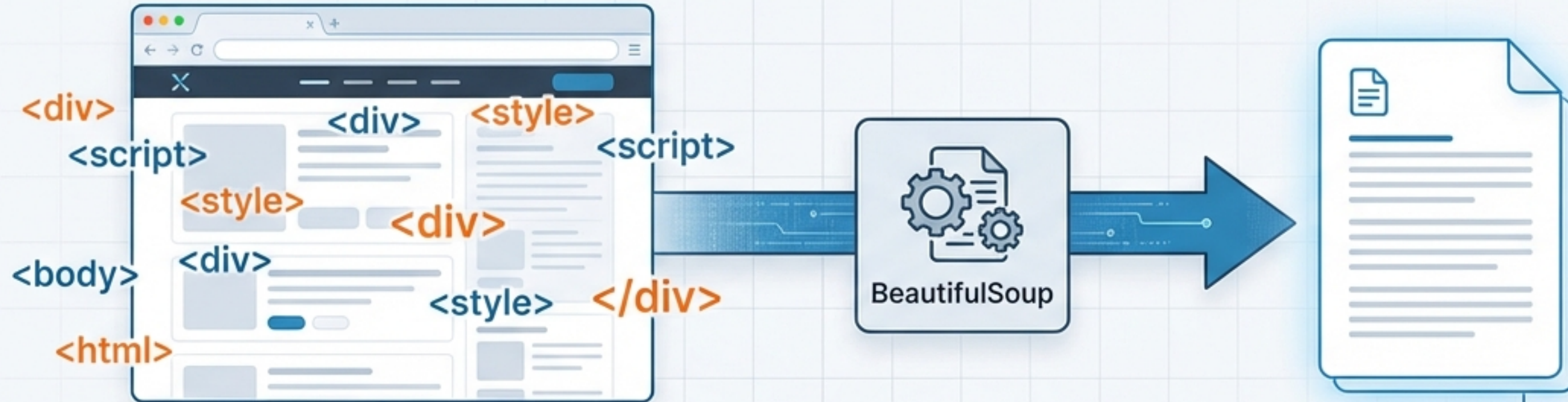
Lazy Loading (`` .lazy_load()``)



```
for doc in loader.lazy_load():  
    process(doc) # One at a time
```

- Returns a Generator
- Low Memory Usage
- Yields individually
- Essential for production scale.

WebBaseLoader scrapes and cleans live web data



```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader('https://campusx.in/blog')
docs = loader.load()
```



Uses `requests` to fetch and `BeautifulSoup` to parse.

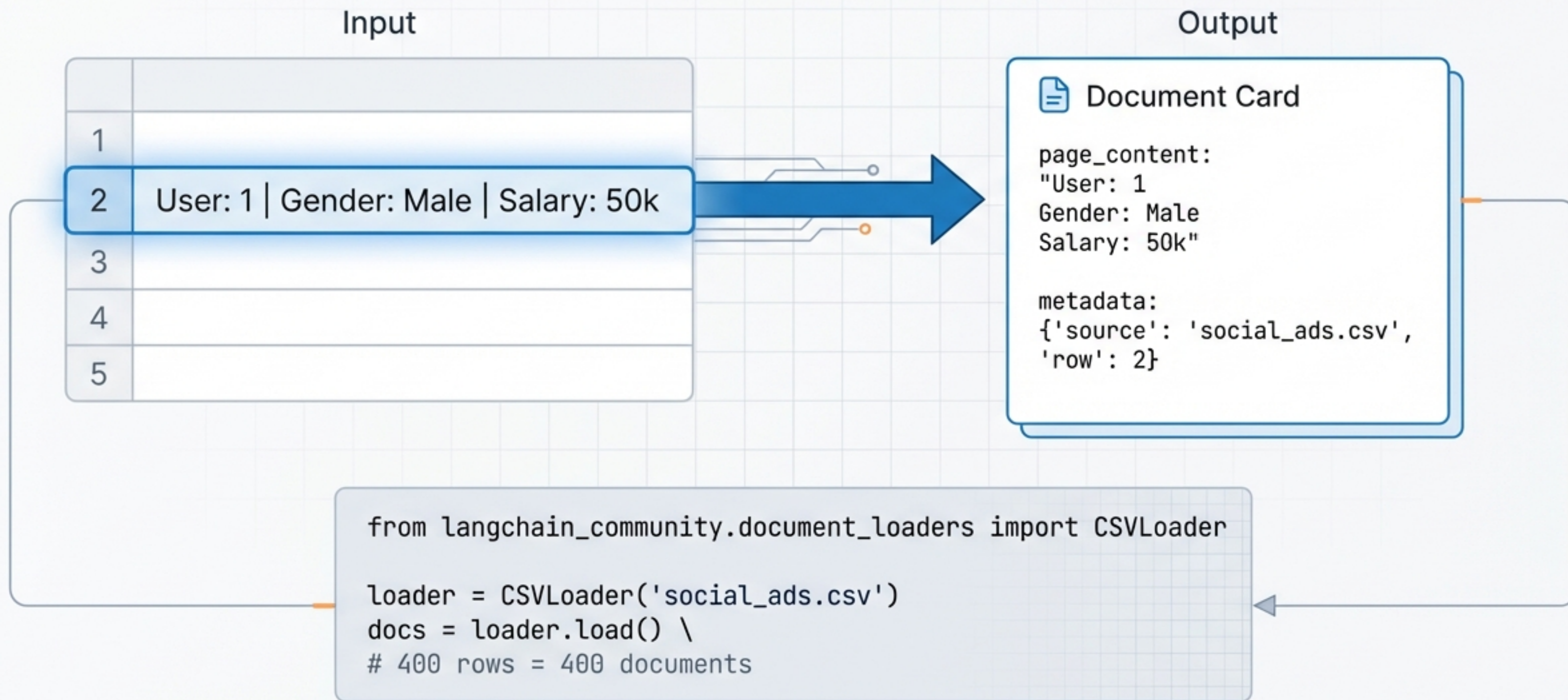


Input: Single URL or List of URLs.

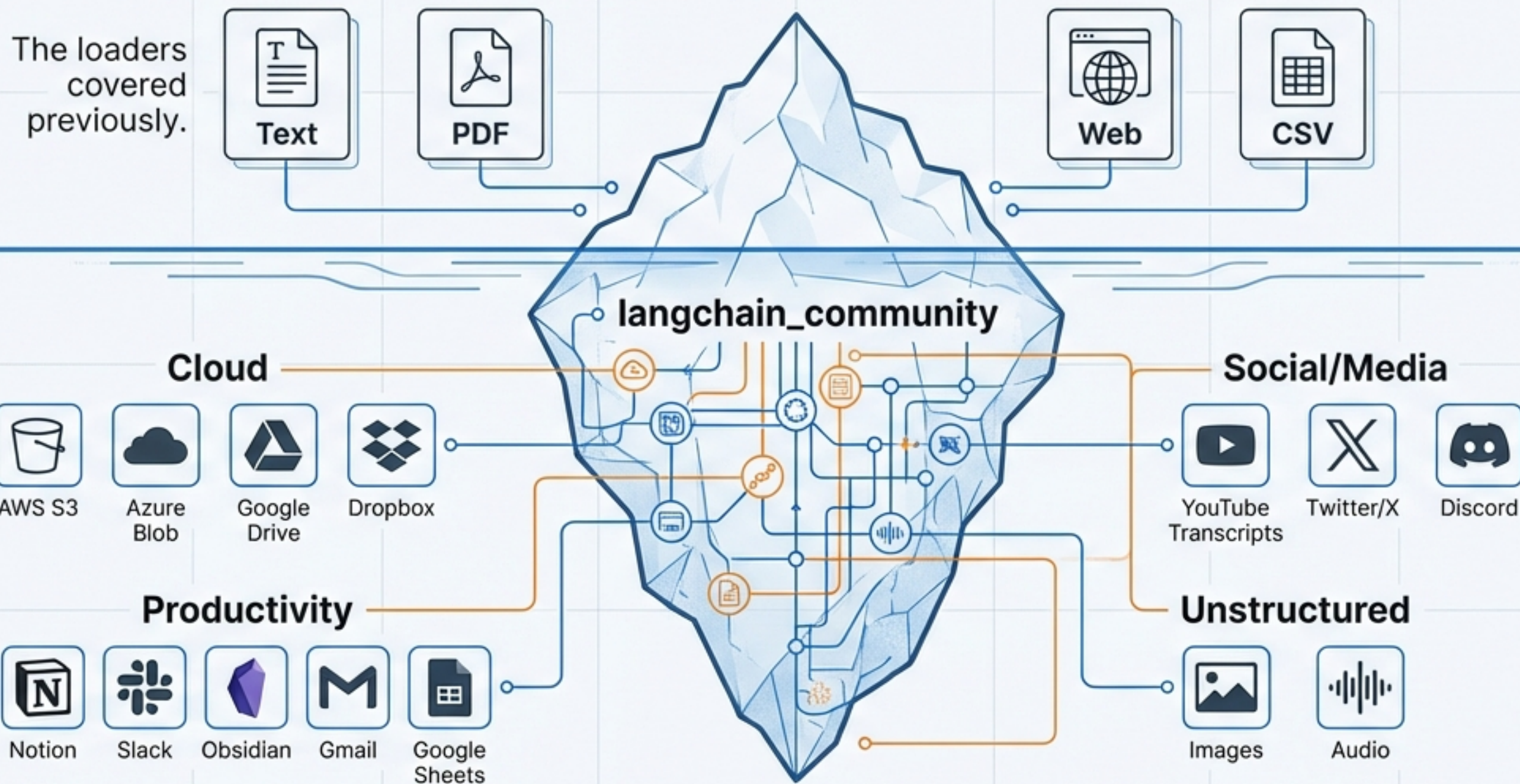


Limitation: Static content only. Use `SeleniumURLLoader` for dynamic/JS sites.

CSVLoader maps structured rows to document context

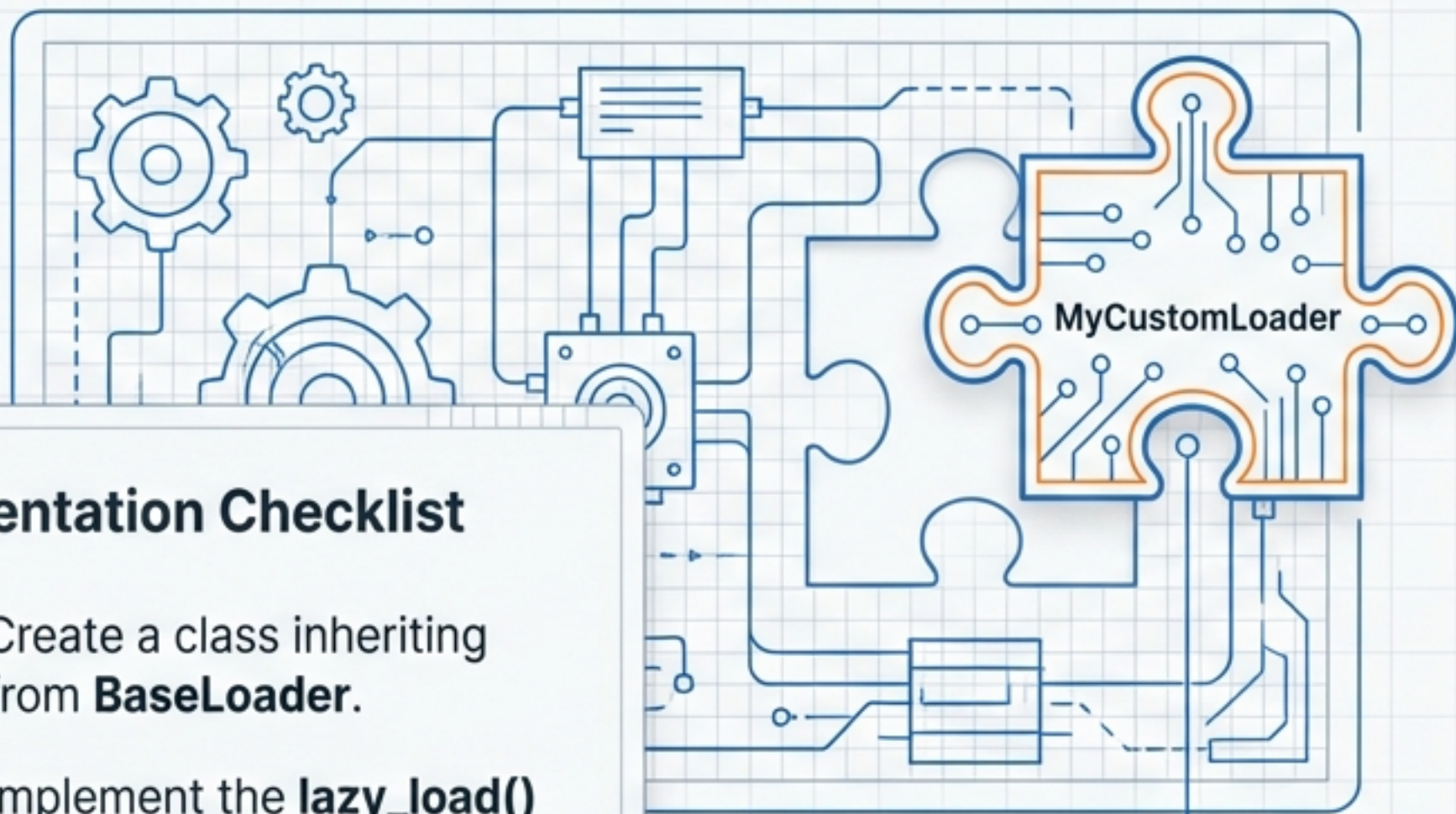


The LangChain Community Ecosystem



Always check the official registry before building from scratch.

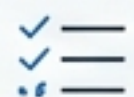
Extending the framework with Custom Loaders



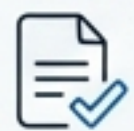
Implementation Checklist



1. Create a class inheriting from **BaseLoader**.



2. Implement the **lazy_load()** method (Mandatory for memory efficiency).



3. Optionally implement **.load()**.

```
from langchain_core.document_loaders
import BaseLoader
```

```
class MyLoader(BaseLoader):
    def lazy_load(self):
        # Yield documents one by one
        yield Document(...)
```


Best Practices for Document Ingestion



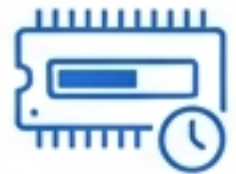
Universal Output

Regardless of source, ensure everything converts to ``Document`` objects (Content + Metadata).



Know the Split

PDFs split by page, CSVs by row, Web by URL. Choose the right unit of text.



Lazy Loading

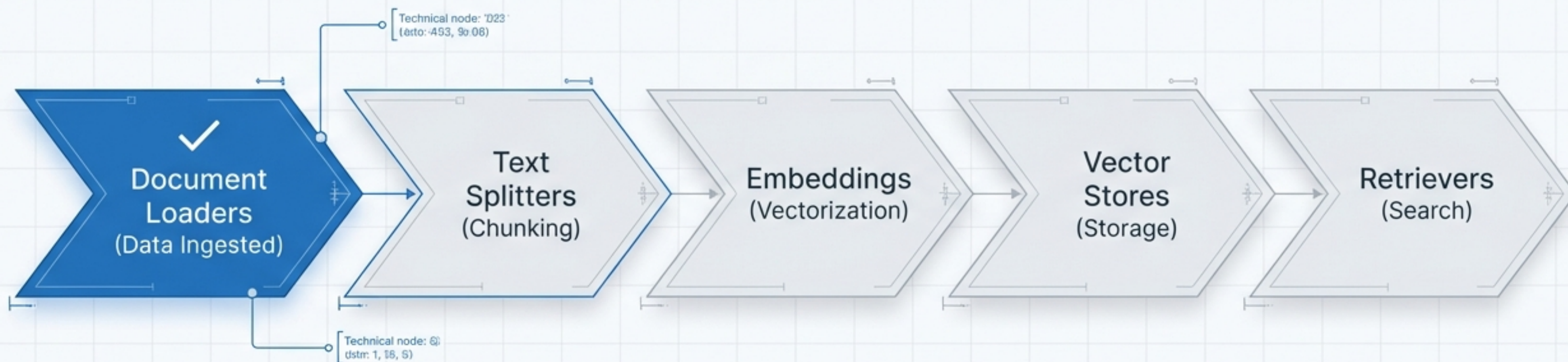
Default to ``.lazy_load()`` for production pipelines to prevent memory crashes.



Community First

Leverage ``.langchain_community`` loaders before engineering custom solutions.

The RAG Pipeline: What comes next?



Now that data is standardized, it is ready to be chunked and embedded.